NIYOKWIZERA Fleury **Le 26/03/2025**

MANIRAKIZA Prudence **A/A : 2024-2025**

BIGIRIMANA Audifax

NDUWAYEZU J. marie

UNIVERSITE DU BURUNDI

INSTITUT DE STATISTIQUE APPLIQUEE

CAMPUS NAMUGERERA

MASTER I

STATISTIQUE APPLIQUEE ET INFORMATIQUE DECISIONNELLE

TRAVAIL PRATIQUE II DU COURS DE BASES DE DONNEES AVANCES

A Requêtes élémentaires

Question 1

Donner l'intégralité de la table personne.

```
mysql> select * from Personne;
+---------+------------+------------+----------+----------+
| id_pers | Prenom     | Nom        | Naisance | Pays     |
+---------+------------+------------+----------+----------+
|       1 | Fleury     | NIYOKWIZERA |     1999 | BURUNDI  |
|       2 | Audifax    | BIGIRIMANA |     1998 | Ghana    |
|       3 | Prudence   | MANIRAKIZA |     1990 | RWANDA   |
|       4 | Jean Marie | NDUWAYEZU  |     1980 | TANZANIE |
|       5 | Nicole     | KIMANA     |     2000 | France   |
|       6 | Aline      | KIRIMANGA  |     1991 | Italie   |
+---------+------------+------------+----------+----------+
6 rows in set (0.00 sec)
```

Question 2

Donner la liste des prénoms des personnes de la table personne.

```
mysql> select Prenom from Personne;
+------------+
| Prenom     |
+------------+
| Fleury     |
| Audifax    |
| Prudence   |
| Jean Marie |
| Nicole     |
| Aline      |
+------------+
6 rows in set (0.00 sec)
```

Question 3

Donner la liste des prénoms des personnes de la table personne,
renommer la colonne Vorname.

```
mysql> select Prenom as Vorname from Personne;
+------------+
| Vorname    |
+------------+
| Fleury     |
| Audifax    |
| Prudence   |
| Jean Marie |
| Nicole     |
| Aline      |
+------------+
6 rows in set (0.00 sec)
```

Question 4

Donner la liste des noms des personnes de la table personne et leur année de naissance.

```
mysql> select Nom,Naisance from Personne;
+-------------+----------+
| Nom         | Naisance |
+-------------+----------+
| NIYOKWIZERA |     1999 |
| BIGIRIMANA  |     1998 |
| MANIRAKIZA  |     1990 |
| NDUWAYEZU   |     1980 |
| KIMANA      |     2000 |
| KIRIMANGA   |     1991 |
+-------------+----------+
6 rows in set (0.00 sec)
```

Question 5

Donner la liste des noms des personnes de la table personne et leur âge dans une colonne age (sachant qu'on est en 2022).

```
mysql> select Nom,(2022 - Naisance) as Age from Personne;
+-------------+-----+
| Nom         | Age |
+-------------+-----+
| NIYOKWIZERA |  23 |
| BIGIRIMANA  |  24 |
| MANIRAKIZA  |  32 |
| NDUWAYEZU   |  42 |
| KIMANA      |  22 |
| KIRIMANGA   |  31 |
+-------------+-----+
6 rows in set (0.00 sec)
```

Question 6

Donner la liste des personnes de la table personne dont l'âge est de plus de 30 ans.

```
mysql> select* from Personne where (2022 - Naisance) > 30;
+---------+------------+------------+----------+----------+
| id_pers | Prenom     | Nom        | Naisance | Pays     |
+---------+------------+------------+----------+----------+
|       3 | Prudence   | MANIRAKIZA |     1990 | RWANDA   |
|       4 | Jean Marie | NDUWAYEZU  |     1980 | TANZANIE |
|       6 | Aline      | KIRIMANGA  |     1991 | Italie   |
+---------+------------+------------+----------+----------+
3 rows in set (0.00 sec)
```

Question 7

Donner la liste des personnes de la table personne nées en France.

```
mysql> select* from Personne where Pays = 'France';
+---------+--------+--------+----------+--------+
| id_pers | Prenom | Nom    | Naisance | Pays   |
+---------+--------+--------+----------+--------+
|       5 | Nicole | KIMANA |     2000 | France |
+---------+--------+--------+----------+--------+
1 row in set (0.00 sec)
```

Question 8

Donner la liste des personnes de la table personne nées avant (au sens large) 1988.

```
mysql> select* from Personne where Naisance < 1988;
+---------+------------+-----------+----------+----------+
| id_pers | Prenom     | Nom       | Naisance | Pays     |
+---------+------------+-----------+----------+----------+
|       4 | Jean Marie | NDUWAYEZU |     1980 | TANZANIE |
+---------+------------+-----------+----------+----------+
1 row in set (0.00 sec)
```

Question 9

Donner la liste des prénoms des personnes de la table personne dont l'identifiant est supérieur ou égal à 3 ou qui ont entre 25 en 30 ans.

```
mysql> select Prenom from Personne where (id_pers >= 3 or (Naisance-2022) between 25 and
30);
+-----------+
| Prenom    |
+-----------+
| Prudence  |
| Jean Marie |
| Nicole    |
| Aline     |
+-----------+
4 rows in set (0.00 sec)
```

Question 10

Donner la liste sans doublons de tous les pays mentionnés par la base de données.

```
mysql> select distinct Pays from Personne;
+----------+
| Pays     |
+----------+
| BURUNDI  |
| Ghana    |
| RWANDA   |
| TANZANIE |
| France   |
| Italie   |
+----------+
6 rows in set (0.00 sec)
```

B Jointures

Question 11

Donner une liste associant les prénoms des personnes de la base avec le code de la capitale de leur pays de naissance.

```
mysql> SELECT p.Prenom, c.code_agglo AS code_capitale FROM personne p JOIN capitale c ON (
ELEC    ->     SELECT code_agglo
    ->     FROM agglomeration a
    ->     WHERE a.lieu = p.Pays
    ->     AND a.code_agglo IN (
    ->         SELECT code_agglo
    ->         FROM capitale
    ->     )
    -> ) = c.code_agglo;
+------------+---------------+
| Prenom     | code_capitale |
+------------+---------------+
| Fleury     |            10 |
| Audifax    |            11 |
| Prudence   |            12 |
| Jean Marie |            13 |
| Nicole     |            14 |
| Aline      |            15 |
+------------+---------------+
6 rows in set (0.00 sec)
```

Question 12

Donner la liste des couples (prénom, code) associant pour chaque personne son prénom et le code d'une agglomération dans laquelle elle a habité.

```
mysql> select Habite.code_agglo ,Personne.Prenom from Habite inner join Personne on Habit
e.id_pers = Personne.id_pers;
+------------+------------+
| code_agglo | Prenom     |
+------------+------------+
|         10 | Fleury     |
|         11 | Audifax    |
|         12 | Prudence   |
|         13 | Jean Marie |
|         14 | Nicole     |
|         15 | Aline      |
+------------+------------+
6 rows in set (0.00 sec)
```

Question 13

Donner les noms et prénoms des personnes ayant habité à Rome.

```
mysql> select Personne.Nom, Personne.Prenom from Personne inner join (Agglomeration inner
 join Habite on Agglomeration.code_agglo = Habite.code_agglo) on Personne.id_pers = Habit
e.id_pers where Agglomeration.Nom = "Rome";
+-----------+--------+
| Nom       | Prenom |
+-----------+--------+
| KIRIMANGA | Aline  |
+-----------+--------+
1 row in set (0.00 sec)
```

Question 14

Donner les noms et prénoms des personnes vivant à Rome en 1999.

```
mysql> select Personne.Nom, Personne.Prenom from Personne inner join (Agglomeration inner
 join Habite on Agglomeration.code_agglo = Habite.code_agglo) on Personne.id_pers = Habit
e.id_pers where Agglomeration.Nom = "Rome" and 1999 between Habite.debut and Habite.fin;
+-----------+--------+
| Nom       | Prenom |
+-----------+--------+
| KIRIMANGA | Aline  |
+-----------+--------+
1 row in set (0.00 sec)
```

Question 15

Donner les noms et prénoms des personnes vivant dans une capitale en 1999.

```
mysql> select Personne.Nom, Personne.Prenom from Personne inner join (Agglomeration inner
 join Habite on Agglomeration.code_agglo = Habite.code_agglo) on Personne.id_pers = Habit
e.id_pers where Agglomeration.Nom in ("Bujumbura", "Dodoma", "Kigali", "Paris", "Rome", "
Kumbuku") and 1999 between Habite.debut and Habite.fin;
+-------------+------------+
| Nom         | Prenom     |
+-------------+------------+
| NIYOKWIZERA | Fleury     |
| BIGIRIMANA  | Audifax    |
| MANIRAKIZA  | Prudence   |
| NDUWAYEZU   | Jean Marie |
| KIRIMANGA   | Aline      |
+-------------+------------+
5 rows in set (0.00 sec)
```

Question 16

Donner une liste de couples d'identifiants représentant les paires de personnes (différentes) ayant vécu à un moment dans la même ville. (On s'assurera donc que la liste résultat ne contient pas de couple de la forme (i,i) et qu'elle ne contient pas à la fois (i,j) et (j,i)).

```
mysql> SELECT DISTINCT
    ->     LEAST(h1.id_pers, h2.id_pers) AS id_pers_1,
    ->     GREATEST(h1.id_pers, h2.id_pers) AS id_pers_2
    -> FROM
    ->     habite h1
    -> JOIN
    ->     habite h2 ON h1.code_agglo = h2.code_agglo
    -> WHERE
    ->     h1.id_pers <> h2.id_pers
    ->     AND (
    ->         (h1.debut BETWEEN h2.debut AND h2.fin)
    ->         OR
    ->         (h2.debut BETWEEN h1.debut AND h1.fin)
    ->     )
    -> GROUP BY
    ->     LEAST(h1.id_pers, h2.id_pers),
    ->     GREATEST(h1.id_pers, h2.id_pers)
    -> ORDER BY
    ->     id_pers_1, id_pers_2;
Empty set (0.01 sec)
```

C Jointure gauche

Question 17

Donner la liste des noms des agglomérations qui ne sont pas des capitales.

```
mysql> select Agglomeration.Nom from Agglomeration left join Habite on Agglomeration.code
_agglo = Habite.code_agglo where Agglomeration.Nom is null;
Empty set (0.00 sec)
```

Question 18

Donner, sans utiliser de fonction d'agrégation, l'agglomération de plus grande population.

```
mysql> select Agglomeration.Nom from Agglomeration left join Habite on Agglomeration.code
_agglo = Habite.code_agglo where Agglomeration.nb_habitat in (select max(nb_habitat) from
 Agglomeration);
+-------+
| Nom   |
+-------+
| Paris |
+-------+
1 row in set (0.01 sec)
```

Question 19

Donner, sans utiliser de fonction d'agrégation, la différence de nombre d'habitants entre l'agglomération la plus peuplée et l'agglomération la moins peuplée.

```
mysql> select max(nb_habitat)-min(nb_habitat) from agglomeration;
+---------------------------------+
| max(nb_habitat)-min(nb_habitat) |
+---------------------------------+
|                        50890000 |
+---------------------------------+
1 row in set (0.00 sec)
```

Question 20

Donner les noms et prénoms des personnes ayant réussi à fuir le système de surveillance, c'est-à-dire des personnes pour lesquelles il manque les informations d'habitation pendant une période de temps précédent 2022.

```
mysql> SELECT DISTINCT p.Prenom, p.Nom
    -> FROM personne p
    -> WHERE NOT EXISTS (
    ->     SELECT 1
    ->     FROM habite h
    ->     WHERE h.id_pers = p.id_pers
    ->     AND h.fin >= '2022'  -- Aucun enregistrement couvrant jusqu'en 2022
    -> )
    -> OR p.id_pers NOT IN (
    ->     SELECT id_pers
    ->     FROM habite
    ->     WHERE debut <= 2022 AND fin >= 2022  -- Aucune période continue incluant 2022
    -> );
+----------+-------------+
| Prenom   | Nom         |
+----------+-------------+
| Fleury   | NIYOKWIZERA |
| Audifax  | BIGIRIMANA  |
| Prudence | MANIRAKIZA  |
+----------+-------------+
3 rows in set (0.01 sec)
```

Question 21

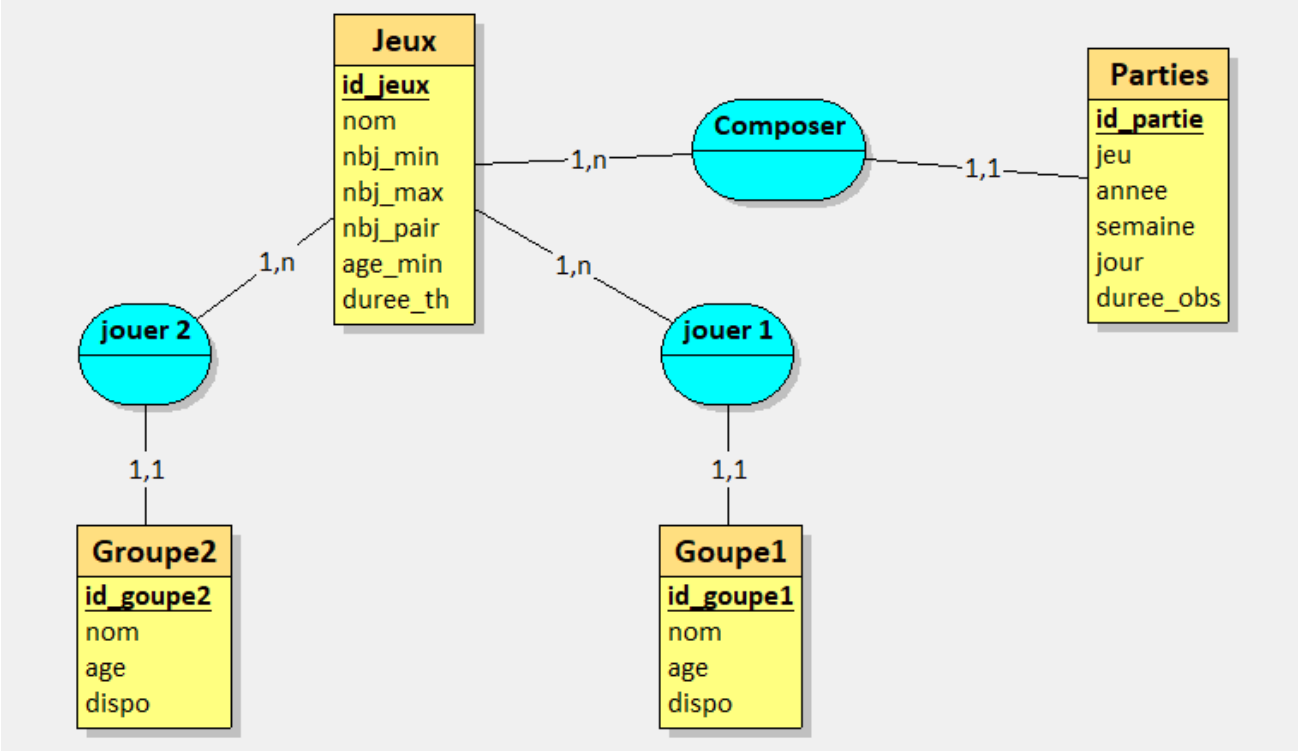
Donner, sans utiliser de fonction d'agrégation, le prénom des personnes ayant vécu dans une ville qui n'est pas une capitale, mais contenant plus d'habitants qu'une des capitales. Avec chaque personne, on donnera aussi la ville en question.

```
mysql> SELECT DISTINCT p.Prenom, a._nom AS Ville
    -> FROM personne p
    -> JOIN habite h ON p.id_pers = h.id_pers
    -> JOIN agglomeration a ON h.code_agglo = a.code_agglo
    -> WHERE a.code_agglo NOT IN (SELECT code_agglo FROM capitale)  -- Ville non-capitale
    -> AND EXISTS (
    ->     SELECT 1
    ->     FROM agglomeration capitale_agglo
    ->     JOIN capitale c ON capitale_agglo.code_agglo = c.code_agglo
    ->     WHERE a._nb_habitant > capitale_agglo._nb_habitant  -- Plus peuplée qu'au moins une
 capitale
    -> );
Empty set (0.00 sec)
```

Exercice 2 Jeux de société

Question 1

Représenter cette base de données par un schéma entité-association.



Question 2

Lister les jeux dont le nombre de joueurs est fixe.

```
mysql> select * from Jeux where nbj_min = nbj_max;
+---------+-----+---------+---------+----------+---------+----------+
| id_jeux | nom | nbj_min | nbj_max | nbj_pair | age_min | duree_th |
+---------+-----+---------+---------+----------+---------+----------+
|      10 | Box |       2 |       2 |        1 |       8 |       36 |
+---------+-----+---------+---------+----------+---------+----------+
1 row in set (0.00 sec)
```

Question 3

Lister les jeux auxquels on peut jouer à 2 joueurs. Combien y en a-t-il ?

```
mysql> select nom from Jeux where nbj_min =2;
+--------+
| nom    |
+--------+
| Tennis |
| Box    |
+--------+
2 rows in set (0.00 sec)

mysql> select count(nom) from Jeux where nbj_min =2;
+------------+
| count(nom) |
+------------+
|          2 |
+------------+
1 row in set (0.00 sec)
```

Question 4

Lister les jeux auxquels on peut jouer à 5 joueurs. Quelle est leur durée moyenne ?

```
mysql> select nom from Jeux where nbj_min =5;
Empty set (0.00 sec)

mysql> select avg(duree_th) from Jeux where nbj_min =5;
+---------------+
| avg(duree_th) |
+---------------+
|          NULL |
+---------------+
1 row in set (0.01 sec)
```

Question 5

Lister les jeux auxquels peut jouer un enfant de 5 ans.

```
mysql> select nom from Jeux where age_min =5;
+--------+
| nom    |
+--------+
| Gold   |
| Karate |
+--------+
2 rows in set (0.00 sec)
```

Question 6

Combien de parties ont été jouées entre 2000 et 2005 ? Combien de jeux différents ont été joués entre 2000 et 2005 ?

```
mysql> select count(id_partie) from Parties where annee between 2000 and 2005;
+------------------+
| count(id_partie) |
+------------------+
|                4 |
+------------------+
1 row in set (0.00 sec)

mysql> select count(nom) from Jeux inner join Parties on Jeux.id_jeux = Parties.id_jeux w
here annee between 2000 and 2005;
+------------+
| count(nom) |
+------------+
|          4 |
+------------+
1 row in set (0.00 sec)
```

Question 7

D’après la table parties quelle est la durée moyenne d’une partie de uno ?

```
mysql> select avg(duree_obs) from Parties where jeu = "uno";
+----------------+
| avg(duree_obs) |
+----------------+
|        14.0000 |
+----------------+
1 row in set (0.00 sec)
```

Question 8

Établir les durées moyennes de chaque jeu selon la table parties. Présenter ces durées dans une colonne appelée duree_moy.

```
mysql> select jeu, avg(duree_obs) as duree_moy from Parties group by jeu;
+------------+-----------+
| jeu        | duree_moy |
+------------+-----------+
| Football   |   90.0000 |
| Tennis     |   90.0000 |
| VolleyBall |   40.0000 |
| BasketBall |   48.0000 |
| Handball   |   60.0000 |
| Neball     |   60.0000 |
| uno        |   14.0000 |
| Gold       |  300.0000 |
| Karate     |    3.0000 |
| Box        |   36.0000 |
+------------+-----------+
10 rows in set (0.00 sec)
```

Question 9

Lister les jeux pour lesquels la durée moyenne observée dans la table parties affiche un écart relatif à la durée théorique de plus de 10%.

```
mysql> SELECT j.nom,
    ->        j.duree_th AS duree_theorique,
    ->        AVG(p.duree_obs) AS duree_moyenne_observee,
    ->        ROUND(ABS(AVG(p.duree_obs) - j.duree_th) / j.duree_th * 100, 2) AS ecart_pourcen
tage
    -> FROM Jeux j
    -> JOIN Parties p ON j.id_jeux = p.id_jeux
UP BY    -> GROUP BY j.id_jeux, j.nom, j.duree_th
    -> HAVING ABS(AVG(p.duree_obs) - j.duree_th) / j.duree_th * 100 > 10
    -> ORDER BY ecart_pourcentage DESC;
Empty set (0.01 sec)
```

Question 10

Combien de parties ont été réalisées dans les 18 deniers mois (i.e. les 78 dernières semaines) ? Disons qu'on est la semaine 21 de l'année 2022. On pourra d'abord caluler le nombre de semaines passées depuis chaque partie.

```
mysql> SELECT COUNT(*) AS nb_parties_recentes
    -> FROM Parties
    -> WHERE
    ->     (annee = 2022 AND semaine BETWEEN 21-78+1 AND 21) OR
    ->     (annee = 2021 AND semaine >= 21 - (78 - (52 - 21)));
+--------------------+
| nb_parties_recentes |
+--------------------+
|                  0 |
+--------------------+
1 row in set (0.00 sec)
```

Question 11

À quelle date faut-il remonter pour dire qu'on a fait 10 parties depuis ?

```
mysql> SELECT p.annee, p.semaine
    -> FROM Parties p
    -> ORDER BY p.annee DESC, p.semaine DESC
    -> LIMIT 1 OFFSET 9;
+-------+---------+
| annee | semaine |
+-------+---------+
|  2001 |      35 |
+-------+---------+
1 row in set (0.01 sec)
```

Question 12

À quels jeux peut jouer le groupe de personnes décrit par la table groupe1 sans tenir compte de leur disponibilité ? Même question pour le groupe décrit par groupe2.

```
mysql> SELECT DISTINCT j.nom
    -> FROM Jeux j
    -> JOIN Goupe1 g ON j.id_jeux = g.id_jeux
    -> WHERE g.age >= j.age_min;
+------------+
| nom        |
+------------+
| Football   |
| Tennis     |
| VolleyBall |
| BasketBall |
| Handball   |
| Neball     |
| uno        |
| Karate     |
+------------+
8 rows in set (0.00 sec)
```

```
mysql> SELECT DISTINCT j.nom
    -> FROM Jeux j
    -> JOIN Groupe2 g ON j.id_jeux = g.id_jeux
    -> WHERE g.age >= j.age_min;
+------------+
| nom        |
+------------+
| Football   |
| Tennis     |
| VolleyBall |
| BasketBall |
| Handball   |
| Neball     |
| uno        |
| Karate     |
+------------+
8 rows in set (0.00 sec)
```

Question 13

Même question en tenant compte de leur disponibilité cette fois ? pour le groupe décrit par groupe2.

```
mysql> SELECT DISTINCT j.nom
    -> FROM Jeux j
    -> JOIN Groupe2 g ON j.id_jeux = g.id_jeux
    -> WHERE g.age >= j.age_min
    -> AND g.dispo >= j.duree_th;
+------------+
| nom        |
+------------+
| Football   |
| Tennis     |
| VolleyBall |
| BasketBall |
| Handball   |
| Neball     |
| uno        |
| Karate     |
+------------+
8 rows in set (0.00 sec)
```

Exercice 3 Transports en commun

A Requêtes simples

Question 1

Donner la liste des noms des lignes de transport.

```
mysql> use TransportDB;
Database changed
mysql> select nom from lignes;
+----------------+
| nom            |
+----------------+
| 7b             |
| LIGNE H        |
| 8              |
| 13             |
| CDGVAL         |
| FUNICULAIRE... |
+----------------+
6 rows in set (0.00 sec)
```

Question 2

Donner l'ensemble (sans doublons donc) des exploitants.

```
mysql> SELECT DISTINCT exploitant from lignes;
+------------+
| exploitant |
+------------+
| RATP       |
| SNCF       |
| Transdev   |
+------------+
3 rows in set (0.01 sec)
```

Question 3

Donner la liste des noms des lignes exploitées par la SNCF.

```
mysql> SELECT nom FROM lignes WHERE exploitant = 'SNCF';
Empty set (0.00 sec)
```

Question 4

Donner l'ensemble (sans doublons donc) des modes de transports qui sont exploités par la SNCF.

```
mysql> SELECT DISTINCT mode FROM lignes WHERE exploitant = 'SNCF';
Empty set (0.00 sec)
```

Question 5

Donner l'ensemble des identifiants des stations dont la fréquentation annuelle est entre 1000000 et 2000000.

```
mysql> SELECT DISTINCT f.id_stations FROM frequentations f WHERE f.nb_entrees BETWEEN 100
0000 AND 2000000;
Empty set (0.01 sec)
```

B Requêtes avec tri

Question 6

Donner l'ensemble des noms des stations de Métro triées par ordre alphabétique.

```
mysql> SELECT nom FROM stations WHERE mode = 'Metro'ORDER BY nom;
+--------+
| nom    |
+--------+
| Gaîté  |
+--------+
1 row in set (0.01 sec)
```

Question 7

Lister par fréquentations décroissantes les stations dont on sait que la fréquentation annuelle est supérieure à 10000000 entrées. Les stations sont ici données par leur identifiant.

```
mysql> SELECT id_stations, nb_entrees FROM frequentations WHERE nb_entrees > 10000000 ORD
ER BY nb_entrees DESC;
Empty set (0.00 sec)
```

C Requêtes avec agrégation

Question 8

Donner le nombre total d'entrées annuelles comptabilisées sur l'ensemble du réseau. Nommer le résultat somme.

```
mysql> SELECT SUM(nb_entrees) AS somme FROM frequentations;
+---------+
| somme   |
+---------+
| 2995369 |
+---------+
1 row in set (0.00 sec)
```

D Requêtes avec jointures

Question 9

Donner l'ensemble des noms de stations de métro par ordre décroissant de fréquentations annuelles.

```
mysql> SELECT s.nom, f.nb_entrees FROM stations s JOIN frequentations f ON s.id_stations
= f.id_stations WHERE s.mode = 'Metro' ORDER BY f.nb_entrees DESC;
+--------+------------+
| nom    | nb_entrees |
+--------+------------+
| Gaîté  |    2995328 |
+--------+------------+
1 row in set (0.01 sec)
```

Question 10

Donner sans redondance l'ensemble des exploitants des lignes dont un arrêt est un terminus. Vous devriez obtenir tous les exploitants.

```
mysql> SELECT DISTINCT l.exploitant FROM lignes l JOIN arrets a ON l.id_lignes = a.id_lig
nes;
+------------+
| exploitant |
+------------+
| RATP       |
| SNCF       |
+------------+
2 rows in set (0.00 sec)
```

Question 11

Donner l'ensemble des stations de la ligne de métro 13.

```
mysql> SELECT s.nom FROM stations s JOIN arrets a ON s.id_stations = a.id_stations JOIN l
ignes l ON a.id_lignes = l.id_lignes WHERE l.nom = '13' AND l.mode = 'Metro';
Empty set (0.00 sec)

mysql>
```

Question 12

Donner une requête permettant le calcul d'un tableau contenant uniquement la station de plus haute et la station de plus basse fréquentation parmi celles dont la fréquentation est connue.

```
mysql> (SELECT s.nom, f.nb_entrees, 'Max' AS type FROM stations s JOIN frequentations f O
N s.id_stations = f.id_stations ORDER BY f.nb_entrees DESC LIMIT 1)
    -> UNION (SELECT s.nom, f.nb_entrees, 'Min' AS type FROM stations s JOIN frequentatio
ns f ON s.id_stations = f.id_stations ORDER BY f.nb_entrees ASC LIMIT 1);
+---------------------+------------+------+
| nom                 | nb_entrees | type |
+---------------------+------------+------+
| Gaîté               |    2995328 | Max  |
| Châtillon-Montrouge |         41 | Min  |
+---------------------+------------+------+
2 rows in set (0.01 sec)
```

Question 13

Donner l'ensemble des noms des stations dont le nombre d'entrées annuelles est supérieur à la moyenne.

```
mysql> SELECT s.nom, f.nb_entrees
    -> FROM stations s
    -> JOIN frequentations f ON s.id_stations = f.id_stations
    -> WHERE f.nb_entrees > (SELECT AVG(nb_entrees) FROM frequentations);
+--------+------------+
| nom    | nb_entrees |
+--------+------------+
| Gaîté  |    2995328 |
+--------+------------+
1 row in set (0.01 sec)
```

E Requêtes bonus

Question 14

1. Donner l'ensemble des noms des stations qui sont le terminus d'une ligne de métro (comprendre : il existe une ligne de métro passant à cette station, et dont elle est un terminus).

```
mysql> SELECT DISTINCT s.nom
    -> FROM stations s
    -> JOIN arrets a ON s.id_stations = a.id_stations
    -> JOIN lignes l ON a.id_lignes = l.id_lignes
    -> WHERE l.mode = 'Metro' AND a.id_terminus IS NOT NULL;
+----------------------+
| nom                  |
+----------------------+
| Châtillon-Montrouge  |
+----------------------+
1 row in set (0.00 sec)
```

2. Donner l'ensemble des noms des stations de métro qui ne sont pas seulement un terminus pour les lignes de métro (comprendre : il existe une ligne de métro passant à cette station dont elle n'est pas un terminus).

```
mysql> SELECT DISTINCT s.nom
    -> FROM stations s
    -> JOIN arrets a ON s.id_stations = a.id_stations
    -> JOIN lignes l ON a.id_lignes = l.id_lignes
    -> WHERE l.mode = 'Metro' AND a.id_terminus IS NULL;
Empty set (0.00 sec)

mysql>
```

3. Donner l'ensemble des noms des stations qui sont le terminus d'une ligne de métro mais pas le terminus de toutes les lignes qui s'y arêtent.

```
mysql> SELECT s.nom
    -> FROM stations s
    -> WHERE EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NOT NULL
    -> )
    -> AND EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NULL
    -> );
Empty set (0.00 sec)
```

Question 15

1. Donner l'ensemble des noms des stations qui ne sont le terminus d'aucune ligne de métro qu'elles desservent (comprendre : pour toute ligne de métro que dessert cette station, elle n'en est pas un terminus).

```
mysql> SELECT s.nom
    -> FROM stations s
    -> WHERE NOT EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NOT NULL
    -> )
    -> AND EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s.id_stations
    ->     AND l.mode = 'Metro'
    -> );
Empty set (0.01 sec)
```

2. Donner l'ensemble des noms des stations qui sont le terminus de toutes les lignes de métro qui la desservent.

```
mysql> SELECT s.nom
    -> FROM stations s
    -> WHERE NOT EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NULL
    -> )
    -> AND EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s.id_stations
    ->     AND l.mode = 'Metro'
    -> );
+----------------------+
| nom                  |
+----------------------+
| Châtillon-Montrouge  |
+----------------------+
1 row in set (0.01 sec)
```

3. Vérifier qu'aucune station ne vérifie les deux contraintes ci-dessu

```
mysql> SELECT 'Aucune station ne vérifie les deux contraintes' AS verification
    -> WHERE NOT EXISTS (
    ->     SELECT 1 FROM stations s
    ->     WHERE EXISTS (
    ->         SELECT 1 FROM arrets a
    ->         JOIN lignes l ON a.id_lignes = l.id_lignes
    ->         WHERE a.id_stations = s.id_stations
    ->         AND l.mode = 'Metro'
    ->         AND a.id_terminus IS NOT NULL
    ->     )
    ->     AND EXISTS (
    ->         SELECT 1 FROM arrets a
    ->         JOIN lignes l ON a.id_lignes = l.id_lignes
    ->         WHERE a.id_stations = s.id_stations
    ->         AND l.mode = 'Metro'
    ->         AND a.id_terminus IS NULL
    ->     )
    -> );
+-----------------------------------------------+
| verification                                  |
+-----------------------------------------------+
| Aucune station ne vérifie les deux contraintes |
+-----------------------------------------------+
1 row in set (0.00 sec)
```

4. Trouver cependant l'unique paire de stations de métro (a) et (b), ayant le même nom mais pas le même identifiant2 et telle que la station (a) vérifie le point 1 et la station (b) le point 2 de la base.

```
mysql> SELECT s1.nom, s1.id_stations AS id1, s2.id_stations AS id2
    -> FROM stations s1
    -> JOIN stations s2 ON s1.nom = s2.nom AND s1.id_stations < s2.id_stations
    -> WHERE EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s1.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NULL
    -> )
    -> AND NOT EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s1.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NOT NULL
    -> )
    -> AND EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s2.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NOT NULL
    -> )
    -> AND NOT EXISTS (
    ->     SELECT 1 FROM arrets a
    ->     JOIN lignes l ON a.id_lignes = l.id_lignes
    ->     WHERE a.id_stations = s2.id_stations
    ->     AND l.mode = 'Metro'
    ->     AND a.id_terminus IS NOT NULL
    -> );
Empty set (0.01 sec)
```